

SAGE: IRL Extension

Mahady H Rayhan, Gabriel R. Edwards, Ashish Pandey, Vani Seth, Nilesh Salvi,
Noah Glaser, Marisa Chrysochoou, Prasad Calyam

University of Missouri - Columbia, USA

Email: {mhr6wb, gre4bt, apfd6, vs4ky, salvin, noah.glaser, mczhd, calyamp}@missouri.edu

Abstract—Public Large Language Models (LLMs) are being increasingly explored for aiding students to learn difficult concepts in STEM education. However, public LLM-based chatbots do not provide course-regulated answers that are important in a tutoring context, where instructors do not want students to work with chatbots on out-of-scope material, seek answers that hinder learning, or to bypass curricular sequencing. In this paper, we present “SAGE” (Student-focused Adaptive Guidance Engine) as part of an intelligent tutoring framework that can enforce instructor-regulated learning through goal-oriented and learning state based content access controls to ensure scaffolded, curricular sequencing and step-by-step guidance of students. SAGE access control features a three-layer gatekeeper mechanism that combines role-based, attribute-based, and cognitive state-based controls built on top of a knowledge graph of learner state and related course content. We evaluate SAGE in an exemplar programming course scenario and compare its performance against state-of-the-art LLM baselines using a custom educational benchmark. Experimental results on the benchmark show that prompt-based tutors (42%) perform similar to SAGE (44%) on metrics of helpfulness owing to prompt engineering. However, they frequently violate instructor policy boundaries, with SAGE providing 90% security for curriculum access control policies compared to 30% for prompt-based tutors.

Index Terms—Human-AI Teaming, Intelligent Tutoring, Knowledge Graphs, Cognitive Scaffolding, Access Control

I. INTRODUCTION

The rapid proliferation of LLMs has transformed the landscape of Intelligent Tutoring Systems (ITS). Unlike earlier rule-based or scripted tutors, public LLMs such as ChatGPT can explain concepts, debug code, and synthesize examples in real time, creating the promise of scalable, personalized instruction for millions of learners [1]. At the same time, their widespread use in academic settings has raised unprecedented concerns about academic integrity, over-reliance on automation, and the hindrance to genuine learning habits/processes [1] [2].

At the heart of this tension is a misalignment between the objectives of general-purpose LLMs and the goals of education, particularly in STEM area course curriculum. LLMs are optimized to be helpful, i.e., to produce fluent, direct and complete answers to user prompts. Effective instruction, however, seeks to cultivate initially Instructor-Regulated Learning (IRL), where instructors actively plan,

This material is based upon work supported by the National Science Foundation (NSF) under Award No.: EEC-2449869. Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the author(s) and do not necessarily reflect the views of the NSF.

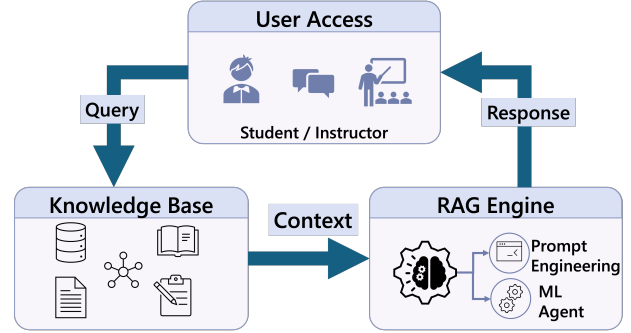


Fig. 1: Illustration of tutoring system that uses a conventional RAG approach.

monitor and assess student learning. However, ultimately instruction could serve students’ Self-Regulated Learning (SRL), in which learners actively plan, monitor, and reflect on their own learning. Zimmerman’s Cyclical Model [3] characterizes SRL as an iterative loop across *forethought* (planning and goal setting), *performance* (strategy use and monitoring), and *self-reflection* (self-evaluation and adaptation). When an LLM provides an end-to-end solutions to a task such as returning a full C language program in response to “How do I reverse an array?”, it risks collapsing this learning cycle because of passive consumption rather than active cognitive engagement of the student who may or may not have adequate background on the underlying concepts.

In addition, current AI-Tutoring systems connect LLMs to external databases through Retrieval-Augmented Generation (RAG) [4] to improve accuracy. However, in most existing systems this integration is treated purely as a search task. As illustrated in **Figure 1**, a conventional “open” RAG pipeline retrieves the most semantically similar content regardless of whether it corresponds to a practice problem, an instructor solution key, or an advanced topic the student has not yet mastered. The retrieved material is then directly injected into the LLMs context [4]. From an educational perspective, this design disregards curriculum structure and learners’ readiness, enabling students to bypass the productive mental struggle necessary for learning [2]. From a Human Machine Systems i.e., ITS perspective, this unrestricted access introduces a security vulnerability: sensitive instructional materials, viz. answer keys or exam materials can be exposed through ordinary dialogue [5]. Thus, what appears as a retrieval design choice simultaneously becomes a pedagogical failure and a instruction policy breach.

Crucially, prior research does not delineate learning goals and security measures as separate issues. Educational practice emphasizes *soft alignment* through prompts that ask the model to “act as a tutor” [1] [2], while security mechanisms emphasize *hard restrictions* that block access to protected data [5]. This separation leads to undesired consequences: prompts can be ignored or manipulated by students seeking answers, while rigid blocking mechanisms cannot distinguish between legitimate scaffolding and cheating. We argue that in AI-assisted pedagogical systems, there is a gap in the state-of-the-art in terms of understanding how to configure access control rules by having curriculum structure being a critical part of the system’s security posture.

Our prior system, SAGE [?], addressed the instructor’s side of this problem. Rather than relying on an LLM to refuse requests through prompt engineering, SAGE enforces instructional policy on a standard RAG pipeline through a three-layer Safety Gatekeeper placed between the learner and the knowledge base: (i) Role-Based Access Control (RBAC), which prevents access to unauthorized files; (ii) Attribute-Based Access Control (ABAC), which aligns content access with curriculum stage and learner progress; and (iii) Cognitive Access Control (CAC), which infers learner intent and constrains the response type, favoring hints and scaffolding over full solutions. Composed over a knowledge graph of learner state and course content, this gatekeeper reached 90% security and 80% curriculum compliance while remaining competitive in helpfulness against prompt-based tutors [?].

SAGE thus provides dependable access control over a set of topics the learner is treated as having mastered. It supplies no principled mechanism, however, for deciding what belongs in that set: membership is conferred by course completion or instructor status rather than earned through evidence of learning. The attribute on which every access decision turns—the learner’s mastered set—is therefore asserted rather than verified. The present article supplies exactly that missing mechanism and feeds its output directly into SAGE’s existing gate.

Deciding what a learner has genuinely mastered is harder than it first appears, because mastery of a programming concept is many-sided: recalling what a pointer is, tracing by hand how one behaves in a loop, and writing correct code that uses it are distinct competencies, and a learner may hold one without the others. A tutor that certifies mastery from a single kind of evidence multiple-choice recall alone, say over-credits learners who have never demonstrated the rest. We call this the *Evidence Diversity Problem*: the systematic over-crediting that arises when heterogeneous evidence is collapsed into one mastery estimate. It is a concrete instance of construct under-representation, and it is especially damaging under instructor-regulated learning, where an inflated mastery record silently corrupts the prerequisite relations the course enforces admitting a learner to a downstream topic on the strength of competence they were never shown to possess.

In this article, we extend SAGE with a learner model that turns a stream of mixed interactions into a competency-certified record of mastered concepts. For each concept the model follows three cognitive levels rather than one, routes each observation to the estimate for its own evidence type, updates those estimates online as the learner works, and certifies a concept as mastered only when all three levels are satisfied at once. Concretely, this article makes the following contributions:

- We name and define the *Evidence Diversity Problem* the over-crediting that arises when a tutor judges mastery from a single kind of evidence and show why it is especially harmful when an instructor governs progression, since an inflated mastery record corrupts the prerequisites the course depends on.
- We introduce a *factored, multi-skill knowledge-tracing learner model* that routes each observation to the estimate for its evidence type, updates online, and grants mastery only under a conjunctive rule over all three levels, producing a mastered set that an instructor-regulated gate can consume directly.
- We prove an *Evidence Diversity Guarantee*: a concept cannot be certified unless the system has observed at least $n_{\min}$ correct observations of *every* required evidence type a property of the architecture itself, independent of the calibrated parameter values and we complement it with quantitative bounds on false certification.
- We integrate this model into SAGE so that its prerequisite gate acts on *earned rather than asserted* mastery, and add a mastery-conditioned response-adaptation policy that fades scaffolding as mastery grows, together with an instructor risk dashboard that surfaces at-risk learners for intervention.

Our evaluation proceeds in two parts. We first extend the benchmark evaluation of SAGE security compliance, curriculum compliance, and pedagogical quality against prompt-based LLM baselines with studies that isolate the learner model itself: how often the system would have certified a concept from a single evidence type, and how closely its mastery judgments track independently held-out performance. We then deploy the extended system in a live introductory C-programming course [PLACEHOLDER: under IRB approval, $N \approx \dots$] and report a usability study of the instructor dashboard and the adaptive tutor in authentic use.

The remainder of this article is organized as follows. Section ?? reviews related work on instructor-regulated tutoring, knowledge tracing, and cognitive diagnosis. Section ?? recaps the inherited SAGE architecture and situates the learner model within it. Section ?? formalizes the factored model, the certification rule, and the Evidence Diversity Guarantee, and describes the adaptation policy, supporting components, and instructor dashboard. Section ?? presents the evaluation, including the classroom study, and Section ?? concludes.

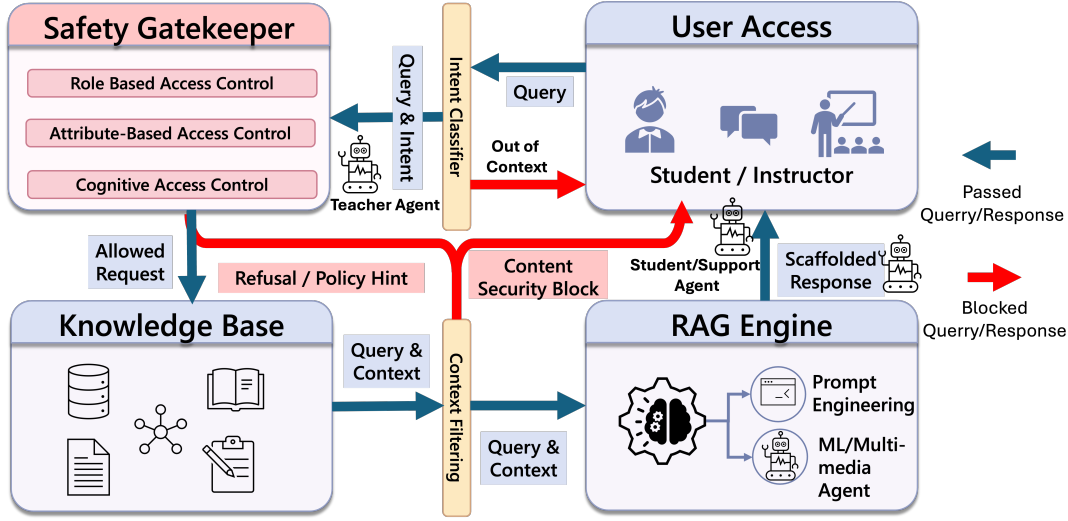


Fig. 2: Overview of SAGE for intelligent tutoring that shows the interaction flow and components for meeting learning goals and enforcing security.

II. RELATED WORK

A. Intelligent Tutoring Systems

Regulated learning is a critical component for academic success, traditionally viewed through Zimmerman’s cyclical model [3]. It consists of learnable processes, such as setting goals and self-monitoring, that are heavily influenced by instructional scaffolding and feedback [6] [7]. Early ITS utilized rule-based pedagogical policies to explicitly prompt learning behaviors such as help-seeking and metacognitive monitoring. While these scaffolds effectively improve learning outcomes and regulatory skills [8], [9], their reliance on manual authoring and rigid domain modeling limits scalability. Conversely, LLMs offer scalable, personalized instruction through natural-language interaction [1]. However, the shift toward generative AI-based tutors necessitates new mechanisms to ensure these systems actively support, rather than bypass, a learner’s internal regulatory processes.

Further, recent studies caution that general-purpose LLMs are not inherently aligned with pedagogical objectives as they prioritize providing complete answers over instructional value [1]. This tendency can allow learners to bypass the productive struggle and metacognitive engagement essential for instructor- or self-regulated learning. Empirical evidence also confirms that the pedagogical impact of AI chatbots depends heavily on interaction design. Towards this, a meta-analysis indicates that learning outcomes improve when interactions provide guided support; conversely, unguided answer-giving can negatively affect higher-order problem-solving [2]. Moreover, systematic reviews suggest that while LLMs enhance accessibility, they risk fostering a reliance on the model that diminishes a learner’s internal regulatory processes when deployed without explicit instructional constraints [10]. We address such concerns in our work through investigations on how interaction structures can be designed to scaffold planning, monitoring, and reflection behaviors,

therefore supporting learners’ ability to learn when engaging with general-purpose LLM-based tutors.

B. Retrieval-Augmented Generation in Educational Tutoring

To mitigate hallucinations and ground responses in course materials, many tutoring systems utilize RAG [4]. While RAG improves factual grounding in learning-oriented settings [11], semantic similarity is often pedagogically agnostic. It fails to account for prerequisite structures or curriculum sequencing; consequently, the most “relevant” retrieved document may be an answer key or advanced material that bypasses the necessary learner’s struggle. Beyond pedagogical concerns, the retrieval layer in RAG presents a significant security risk as well. Recent research [5] indicates that retrieved context is a primary leakage channel where prompt injections can manipulate LLMs into revealing sensitive or restricted information once it enters the context window. Since generation-layer safeguards are often probabilistic and bypassable, these risks motivate the enforcement of constraints at the retrieval boundary rather than relying solely on language-level refusals.

Moreover, current tutoring approaches typically treat learning goals and security mechanisms as separate problems. While educational research investigates the potential of LLMs to enhance regulated learning and metacognitive support [1], [2], the security community emphasizes the risks associated with prompt injection and data poisoning in these integrated systems [5]. The disconnect between these domains creates a significant limitation in existing platforms, as the absence of integrated access controls prevents the enforcement of structured, step-by-step guidance when students access RAG implementations for course content. Existing solutions for content moderation often rely on Reinforcement Learning from Human Feedback [12] or explicit regex-based content filters at the generation layer. However, these safeguards do not change underlying LLMs’ access to information; therefore, as long as sensitive content is present

in the retrieval context, there remains a non-zero probability that it will be exfiltrated under adversarial prompting [5].

In addition to the advances in RAG, recent work on GraphRAG that uses knowledge graphs has demonstrated improved controllability by encoding concept dependencies and curriculum structure that flat vector similarity ignores [13]. There has been further evidence that shows using knowledge graphs can enhance interpretability and alignment in sequenced learning domains [14]. However, existing GraphRAG and secure RAG frameworks typically treat retrieval as a neutral preprocessing step, rather than an explicit component of the instructional design. From a security perspective, this limitation has motivated a shift toward restricting LLM access to data, rather than relying solely on behavioral alignment, to mitigate risks such as prompt injection [5]. The above works motivate our Safety Gatekeeper-based approach in which retrieval itself is governed by pedagogical and organizational policy, enabling access to be conditioned on instructional intent and curriculum constraints.

C. Learner Modeling: Knowledge Tracing and Cognitive Diagnosis

Estimating what a student knows from a sequence of responses is the province of *knowledge tracing*. Bayesian Knowledge Tracing (BKT) [?] models each skill as a two-state hidden Markov process and updates a posterior probability of mastery after every response, giving a transparent, online estimate that has anchored mastery-based progression in intelligent tutors for three decades. Deep Knowledge Tracing (DKT) [?] and its successors replace the per-skill Markov assumption with a recurrent network, improving predictive accuracy by sharing statistical strength across skills at the cost of interpretability and a substantial data requirement. For all their differences, both families share a property that matters here: they summarize a learner’s competence on a skill as a *single* scalar, collapsing recall, procedural fluency, and applied problem-solving into one number. A learner who answers multiple-choice questions well but cannot yet write working code is indistinguishable, to such a model, from one who can do both precisely the over-crediting the Evidence Diversity Problem describes, and a textbook case of construct under-representation in measurement terms [?].

Cognitive diagnosis models (CDMs) take the complementary step of separating competence into multiple latent attributes. The DINA model [?], [?], in particular, applies a *conjunctive* rule: a task is answered correctly only if the learner possesses *all* of its required attributes, so mastery of a compound skill demands every component at once. This is the same all-or-nothing intuition our certification rule rests on, and we claim no novelty for it. CDMs, however, are designed for after-the-fact psychometric diagnosis: their attributes are estimated in batch from a fixed response matrix

TABLE I: Capabilities of representative learner-modeling and tutoring approaches. ✓: supported; △: supported only by some variants; ×: not supported.

Capability	BKT	DKT	CDM	SAGE	Ours
Online updating	✓	✓	△	×	✓
Multi-dimensional mastery	×	×	✓	×	✓
Evidence-type routing	×	×	×	×	✓
Conjunctive mastery	×	×	✓	×	✓
Curriculum enforcement	×	×	×	✓	✓
Mastery-conditioned adaptation	×	×	×	×	✓

calibrated against a hand-authored Q-matrix, they do not update online as a student works, and critically for our setting they do not route evidence by *type*, treating all items that load on an attribute as interchangeable. They tell an analyst which attributes a cohort lacks; they do not hand a tutor a running, evidence-diverse mastery signal on which to gate the next interaction.

D. Mastery-Based Adaptation and Expertise Reversal

Once a mastery estimate exists, a tutor must decide how to act on it. The *expertise-reversal effect* [?] establishes that instructional support which helps novices—worked examples, heavy scaffolding becomes redundant or even counterproductive as competence grows, motivating adaptation that fades support in step with mastery and keeps instruction within the learner’s zone of proximal development [?]. Our mastery-conditioned response-adaptation policy (Section ??) applies this principle to an LLM tutor, mapping the certified mastery level to scaffolding depth, response format, and challenge type. We treat this as a faithful implementation of an established principle rather than a new mechanism; its role here is as the consumer that turns the certified signal into pedagogical action.

E. Summary of the Gap

Table I places these strands side by side. The two capabilities the Evidence Diversity Problem demands are each present in prior work, but never together. Knowledge tracing updates a mastery estimate online as a student works, but collapses every kind of evidence into one score. Cognitive diagnosis models separate competence into several dimensions and can require all of them at once, but are built for after-the-fact diagnosis rather than tutoring control, and do not route evidence by type. And instructor-regulated tutors such as SAGE can act on a mastered set but cannot themselves determine what belongs in it. The learner model of this article brings these strands together routing each kind of evidence to its own estimate, updating those estimates online, and granting mastery only when all are met so that the resulting record of mastered topics can be trusted by an instructor-regulated gate.

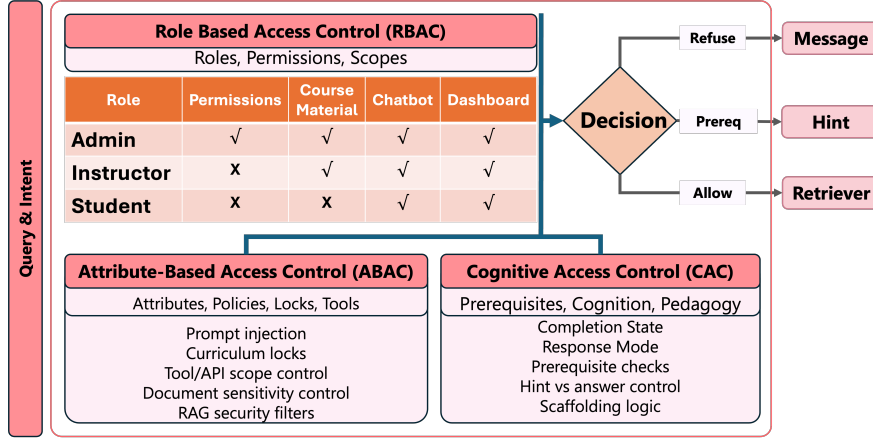


Fig. 3: SAGE security architecture to enforce how the system routes user queries through a Safety Gatekeeper that incorporates semantic analysis and specific access control protocols, including RBAC, ABAC, and CAC.

III. SAGE METHODOLOGY

A fundamental challenge in the deployment of general-purpose LLMs for education is the *alignment gap* between a Large Language Model’s stochastic helpfulness objective and the rigid constraints of an academic curriculum. SAGE addresses this by transforming the LLM tutor into a *stateful component* of an ITS, governed by a deterministic Safety Gatekeeper as shown in **Figure 2** that controls what information is available to the LLM’s context. This design enables the SAGE to respond naturally to user queries while i) preserving the learner’s regulated learning cycle, and ii) strictly enforcing institutional policies regarding assessment content and curriculum pacing. We model the learner session state as:

$$S = \{K, R, C\} \quad (1)$$

where K denotes the set of mastered knowledge nodes (topics), R is the user role (e.g., student or instructor), and C denotes the current cognitive intent (e.g., scaffolding request, solution seeking, or administrative command). Each incoming query q updates S and determines whether access to a candidate document d is permitted. In ITS terms as a human-machine system, the LLM operates as an *actuator* within a closed control loop, where learner actions shape the session state S , thereby constraining what the LLM can retrieve and generate.

A. Dual-Store Knowledge Architecture

SAGE employs a hybrid knowledge architecture that integrates unstructured semantic retrieval with structured curriculum modeling. Unlike flat vector retrieval, a GraphRAG design is used, consisting of two coupled stores: i) A **Vector Database** V (ChromaDB), which indexes chunked instructional content as dense embeddings for semantic similarity retrieval and ii) A **Knowledge Graph** $G = (T, E)$ (Neo4j), where T is the set of topic nodes and E encodes prerequisite relationships in a Directed Acyclic Graph (DAG). Each topic $t \in T$ stores difficulty, instructor-defined learning status, and mastery metadata.

Given a query q , initial candidate chunks are retrieved via:

$$\mathcal{D}_{cand} = \text{topK}(V, \text{embed}(q)). \quad (2)$$

Named entities and topic mentions are mapped to nodes in G . For a topic t , prerequisites are:

$$\text{Prereq}(t) = \{t' \in T \mid (t', t) \in E\}. \quad (3)$$

Given learner knowledge state K , readiness is defined as:

$$\text{Ready}(t, K) = \begin{cases} 1, & \text{if } \text{Prereq}(t) \subseteq K, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Thus retrieval becomes *state-aware*: if q requests “arrays” but the learner lacks proficiency in “loops,” then $\text{Ready}(\text{Arrays}, K) = 0$ and the system redirects toward prerequisite scaffolding rather than providing a full solution. **Human-in-the-Loop Control**: Instructors can modify topic attributes and prerequisite relations within the Knowledge Graph serving as a “human-in-the-loop control”, and these modifications are immediately reflected in the Safety Gatekeeper decisions during subsequent queries as shown in **Figure 3**. An instructor-facing interface is used to allow authorized users (i.e., instructors) to lock or unlock topics, adjust prerequisite relationships, and flag sensitive materials within the Knowledge Graph. These updates propagate immediately to the Safety Gatekeeper, enabling real-time policy enforcement without redeploying LLMs or retraining classifiers. This design supports dynamic curriculum management and allows instructors to intervene when pedagogical goals or assessment policies change.

Thus, SAGE operates over an instructor-controlled course corpus containing lecture notes, practice materials, and restricted assessment artifacts. The corpus is chunked into passages for vector indexing, and each chunk d is annotated with: (i) a type label $\text{type}(d)$ such as lecture, practice, solution, exam key for RBAC; (ii) an associated curriculum topic t when applicable for ABAC; and (iii) a response mode $\text{mode}(d)$ such as conceptual explanation, hint, partial solution, and instructor-only for CAC. These annotations are produced via an instructor-facing interface and validated by spot checks.

B. Safety Gatekeeper and Access Control

SAGE security architecture in the three-layer Safety Gatekeeper is shown in **Figure 3**. Herein, we discuss access control for each candidates query, where the retrieved data chunks are represent by $d \in \mathcal{D}_{cand}$:

Role-Based Access Control (RBAC): Each of the retrieved chunks for the query carries a type label by design such as lecture note, practice solution, exam key, etc. The access to these chunks are permitted if:

$$RBAC(d, R) = \begin{cases} 1, & \text{if role_perm}(R, \text{type}(d)) = \text{allow}, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

For instance, exam answer keys are always denied for role $R = \text{student}$, thus removing sensitive content from possible exposure.

Attribute-Based Access Control (ABAC): Subsequently, if the retrieved chunks d corresponds to topic t , access to them depends on curriculum state and mastery i.e., for example whether the topic is open or not.

$$ABAC(d, K) = \begin{cases} 1, & \text{if status}(t) = \text{Open} \wedge \text{Ready}(t, K) = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

The instructors may lock topics or enforce pacing constraints, ensuring zone-of-proximal-development alignment.

Cognitive Access Control (CAC): We map four anchors into a conservative access decision by checking whether the retrieved document’s *response mode* is compatible with the inferred intent. Towards this, a lightweight intent model infers a coarse tutoring intent label $C \in \{\text{conceptual}, \text{problem_solving}, \text{debugging}, \text{review}\}$:

$$CAC(C, d) = \begin{cases} 1, & \text{if mode}(d) \in \mathcal{M}(C) \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

where $\mathcal{M}(C)$ encodes organizations’ tutoring policy for example, *problem_solving* permits hints or partial code, while instructor-only solution keys remain disallowed for students. This formulation subsumes the binary distinction between *solution-seeking* and *scaffolding-oriented* requests by treating *problem_solving* as the solution-adjacent anchor and the remaining intents as scaffolding-oriented.

Implementation Details: The CAC layer performs low-latency intent classification using the CPU-based sentence embedding model `all-MiniLM-L6-v2` with 384-dimensional embeddings. The intent is inferred via cosine-similarity matching against coarse anchors for conceptual inquiry, problem solving, debugging, and review, with conservative heuristic overrides for security-sensitive queries. Domain-specific entity extraction is handled separately using `gliner_small-v2.1`, a zero-shot span classification model that supports custom labels without retraining. This enables robust identification of programming constructs and error types that are poorly handled by conventional NER methods. CAC decisions are intentionally conservative and composed conjunctively with RBAC and ABAC constraints.

Algorithm 1 SAGE Query Processing

Require: Query q , session state $S = \{K, R, C\}$
Ensure: Tutor response y

```

1: // Semantic analysis
2:  $C \leftarrow \text{IntentClassifier}(q)$ 
3:  $\mathcal{T}_q \leftarrow \text{ExtractTopics}(q)$  ▷ map to KG topics
4:  $\mathcal{D}_{cand} \leftarrow \text{topK}(V, \text{embed}(q))$  ▷ vector retrieval
5: // Prerequisite & curriculum checks
6: for all  $t \in \mathcal{T}_q$  do
7:    $\text{ready}[t] \leftarrow \text{Ready}(t, K)$  ▷ Eq. (5)
8: end for
9: // Safety Gatekeeper filtering
10:  $\mathcal{D}_{allow} \leftarrow \emptyset$ 
11: for all  $d \in \mathcal{D}_{cand}$  do
12:    $r \leftarrow RBAC(d, R)$ 
13:    $a \leftarrow ABAC(d, K)$ 
14:    $c \leftarrow CAC(C, d)$ 
15:   if  $r \wedge a \wedge c = 1$  then
16:      $\mathcal{D}_{allow} \leftarrow \mathcal{D}_{allow} \cup \{d\}$ 
17:   end if
18: end for
19: // No safe context available
20: if  $\mathcal{D}_{allow} = \emptyset$  then
21:   if  $\exists t \in \mathcal{T}_q : \text{ready}[t] = 0$  then
22:     return  $\text{PREREQUISITEHINT}(\mathcal{T}_q, K)$ 
23:   else
24:     return  $\text{REFUSALMESSAGE}(q)$ 
25:   end if
26: end if
27: // Agent selection and generation
28: if  $R = \text{student}$  then
29:    $y \leftarrow \text{SocraticStudentAgent}(q, \mathcal{D}_{allow}, S)$ 
30: else
31:    $y \leftarrow \text{TeacherAdminAgent}(q, \mathcal{D}_{allow}, S)$ 
32: end if
33: return  $y$ 

```

As a result, misclassification can affect the instructional responses, for example level of scaffolding or explanation depth, but does not affect content access or retrieval, which is primarily governed by RBAC and ABAC.

As a result of the three-layered Safety Gatekeeper, a document d is admitted to the LLM context only if

$$\text{Allow}(d \mid S) = RBAC(d, R) \wedge ABAC(d, K) \wedge CAC(C, d) \quad (8)$$

A document is therefore included in the context window iff $\text{Allow}(d \mid S) = 1$. **Algorithm 1** summarizes this query-handling loop, showing how RBAC, ABAC, and CAC are composed to identify the appropriate context window. Once the context is defined, the system delegates control to different pedagogical agents that control information presentation to the learner and provide regulatory control to instructors.

C. Multi-Agent Pedagogical Interaction

We use the term *multi-agent* to denote a set of role-specialized pedagogical agents (student, teacher/admin, and multimedia support) orchestrated by the Safety Gatekeeper, rather than fully autonomous agents with independent goals or learned policies. The system comprises three role-aligned agents:

- 1) *Socratic Student Agent*: promotes self regulated learning (SRL) forethought and performance by generating structured hints, logic plans, and optionally flow diagrams rather than complete solutions.
- 2) *Administrative Teacher Agent*: operates with elevated privileges, enabling curriculum manipulation for example locking/unlocking modules and access to restricted assets via secure function calls.
- 3) *Multimedia Support Agent*: generates visual scaffolds such as state diagrams, memory models, control-flow charts to assist conceptual reasoning and neurodiverse learners.

Together, these agents cover the forethought (planning prompts), performance (scaffolded execution), and self-reflection (post-hoc analysis and feedback) phases of Zimmerman’s regulated learning cycle, making the Safety Gatekeeper a learning-aware controller rather than a generic content filter. All agents work in synchronization with the Safety Gatekeeper and dual-store knowledge architecture, ensuring consistency of instructor-defined curriculum constraints across learning sessions.

D. The Learner Model: Factored Knowledge Tracing

Thus far, the mastered set K has been treated as given. In SAGE as originally formulated, membership in K is conferred by course completion or instructor status: the gate is sound, but the attribute it acts on is asserted. We now specify the learner model that *earns* K from a stream of mixed interactions. Its output is the same object the gate already consumes the set K of Eq. (1), read by $\text{Ready}(t, K)$ in Eq. (4) and by $\text{ABAC}(d, K)$ in Eq. (6)—so the model changes the *provenance* of K without altering a single gate equation. We design the model to three ends: it must update online as the learner works, it must distinguish kinds of evidence rather than pooling them, and it must certify mastery under a rule an instructor-regulated gate can trust.

Mastery of a programming concept is not monolithic: recalling a definition, writing a single correct line, and debugging working logic are distinct competencies. We therefore track each topic $t \in T$ not with one estimate but with three, one per cognitive tier $k \in \{1, 2, 3\}$ corresponding to declarative, procedural, and applied understanding. Each tier maintains an unconstrained Bayesian Knowledge Tracing posterior $\tilde{P}_t^{(k)} \in [0, 1]$, the probability that the learner has mastered topic t at tier k .

E. Conjunctive Certification and the Earned Mastered Set

A topic is certified as mastered only when *every* tier is satisfied at once. Formally, $t \in K$ iff

$$\underbrace{\forall k : \tilde{P}_t^{(k)} \geq \theta_{\text{cert}}}_{\text{all tiers confident}} \wedge \underbrace{\forall k : N_t^{(k)} \geq N_{\min}}_{\text{all tiers evidenced}}, \quad (9)$$

with $\theta_{\text{cert}} = 0.95$ and $N_{\min} = 3$. Certification is evaluated on the raw posteriors; the ceilings $\theta_{\max}^{(k)}$ of Table ?? shape

only the composite score shown to the learner, so an early-stage learner is never displayed high applied mastery even while the underlying posterior is still accumulating. To prevent oscillation at the boundary, certification is governed by Schmitt-trigger hysteresis: once certified, a topic is *decertified* only if some tier decays below $\theta_{\text{dec}} = 0.75 < \theta_{\text{cert}}$.

The certified set $K = \{t \in T : t \text{ satisfies Eq. (9)}\}$ is precisely the mastered set of the session state $S = \{K, R, C\}$ in Eq. (1). It is consumed unchanged by $\text{Ready}(t, K)$ (Eq. (4)) and $\text{ABAC}(d, K)$ (Eq. (6)); the Safety Gatekeeper is not modified. The learner model’s sole effect on access control is to replace an asserted K with an earned one, so that every prerequisite and pacing decision the gate makes now rests on demonstrated, evidence-diverse competence.

F. Mastery-Conditioned Response Adaptation

Certification governs what the tutor may *retrieve*; the same mastery signal also governs how it should *respond*. The expertise-reversal effect [?] holds that the heavy scaffolding which helps a novice becomes redundant, and eventually obstructive, as competence grows. We operationalize this with a policy Π that maps the learner’s mastery on the current topic to a response configuration—scaffolding depth, explanation style, and challenge type. Let $\ell(t) \in \{\text{NOVICE}, \text{DEVELOPING}, \text{PROFICIENT}, \text{REVIEWING}\}$ be the adaptation level for topic t , where the REVIEWING level is entered exactly when t is certified ($t \in K$) and the three remaining levels partition the pre-certification range of the composite mastery estimate. Table II gives Π .

TABLE II: Mastery-conditioned response policy Π . The REVIEWING level coincides with certification.

Level	Scaffolding & style	Challenge
NOVICE	Maximum; real-world analogies, concept in small steps	Guided fill-in-the-blank
DEVELOPING	Moderate; builds on prior known concepts	Multi-concept synthesis
PROFICIENT	Minimal; skips definitions, targets edge cases, memory safety, subtle bugs	Debugging / output prediction
REVIEWING	None; concise refresher only	Quick retention check

Two aspects tie the policy back to the gate. First, when a topic is certified, withholding its solution is no longer pedagogically warranted—the learner has demonstrably mastered it—so the REVIEWING level relaxes the Socratic withholding that CAC (Eq. (7)) applies to uncertified topics, replacing hint-only responses with a brief refresher and a retention check. Certification thus has two coordinated effects: it opens the prerequisite gate for *downstream* topics

through K (Eqs. (4), (6)), and it lifts solution-withholding on the *mastered* topic itself. Second, Π is overridden by two guardrails—a frustration-triggered reframe that suppresses raw code output, and a rushing-triggered concise mode driven by lightweight affective and timing signals logged in the telemetry layer (Sec. III-H).

G. Supporting Components: Retention, Cognitive Load, and Misconceptions

Three further components keep the mastery estimate faithful over time and inform adaptation.

Retention and spaced repetition: Knowledge lapses when unpracticed. Before each update, tier k 's posterior is decayed toward its prior according to the time Δt (in days) since the last interaction on the topic:

$$\tilde{P}_t^{(k)} \leftarrow \tilde{P}_t^{(k)} e^{-\lambda_k \Delta t} + p_{L_0}^{(k)} (1 - e^{-\lambda_k \Delta t}), \quad (10)$$

with per-tier half-lives of 14, 7, and 5 days for the declarative, procedural, and applied tiers. The rate λ_k is not fixed: each successful retrieval updates an SM-2 ease factor $EF \geq 1.3$ from a quality score, and the rate is slowed for well-retained material as $\lambda_k \leftarrow \lambda_k / EF$, so repeatedly demonstrated mastery decays more slowly. Combined with the hysteresis of Eq. (9), this makes K *non-monotone*: a certified topic left unpracticed can decay below θ_{dec} and be decertified, so the gate continues to reflect current rather than historical competence.

Cognitive load: To avoid overwhelming the learner, the model estimates instantaneous load from three telemetry signals, each normalized to $[0, 1]$ —code complexity $\hat{c}$ (an AST-depth proxy), response latency $\hat{\ell}$ (capped at 20 s), and recent error rate $\hat{e}$:

$$C_{\text{load}} = 0.4 \hat{c} + 0.3 \hat{\ell} + 0.3 \hat{e}. \quad (11)$$

High load suppresses complex code generation in favor of plain-language explanation.

Active misconceptions: Recurrent, specific error patterns for example, confusing $=$ with $==$ are tracked per concept with an exponential moving average over the incorrectness indicator $\mathbf{1}[e = 0]$:

$$M_c \leftarrow \lambda_M M_c + (1 - \lambda_M) \mathbf{1}[e = 0], \quad \lambda_M = 0.7. \quad (12)$$

A concept with $M_c \geq 0.25$ is flagged as an active misconception and requires two consecutive correct responses to clear. Active misconceptions are surfaced to the instructor (Sec. III-H) and bias the tutor toward targeted correction.

H. Instructor Dashboard and Telemetry

Every signal above is derived from an append-only telemetry stream that SAGE persists per session: for each concept and tier, the live BKT posteriors, evidence counts $N_t^{(k)}$, and decay rates; a turn log recording the full state vector (code complexity, latency, inferred intent, and affect);

and a prediction log pairing each pre-update posterior with the observed outcome, enabling continuous Brier-score monitoring of the model itself. Because the stream is append-only, it supports offline replay and psychometric re-analysis without disturbing the live model.

This substrate extends SAGE's existing human-in-the-loop control (Sec. ??) from curriculum authoring to learner monitoring. Where instructors could already lock topics and edit prerequisites, the learner model now surfaces, per learner and per topic, certified and decertified mastery, stalled progress, active misconceptions, and elevated frustration or cognitive load, prioritizing learners for attention accordingly. Actions taken through this interface—adjusting pacing, unlocking a prerequisite, flagging a misconception for review—propagate immediately to the gate, closing the loop between what the model observes and what the instructor regulates.

I. Implementation and Runtime Optimization

The system is implemented as a containerized microservices architecture. We use FastAPI to orchestrate the back-end and connect ChromaDB [15] (Vector Database) and Neo4j [16] (Knowledge Graph) to operate as synchronized services. The multi-agent coordination uses a lightweight orchestration framework, with all policy enforcement centralized in the Safety Gatekeeper. To sustain real-time tutoring performance, intent detection and access checks run on CPU-based local LLMs rather than remote LLMs. This design reduces the security overhead to a negligible 135 ms and lowers the Time-to-First-Token from 13.1 s to 2.2 s, demonstrating that rigorous architectural security can co-exist with real-time tutoring requirements. The LLMs used to establish the baseline, such as GPT [17] and Gemini [18] are accessed via their public APIs without fine-tuning or weight modification. SAGE also uses an API-based LLM (Gemini 2.5 flash) [18] as the generator, but routes each query through the Safety Gatekeeper and controlled retrieval pipeline before generation.

A conventional subject course such as Mathematics or Biology can be transformed into our SAGE used as part of an intelligent tutoring framework by structuring the curriculum as a prerequisite graph, embedding course materials into the retrieval store, and defining light-weight access policies across the security layers using existing SAGE APIs. This process requires only course-specific content structuring and policy configuration, while the underlying Tutor architecture remains unchanged, enabling rapid and consistent deployment across different academic domains.

IV. PERFORMANCE EVALUATION

We conduct a multi-dimensional evaluation of our SAGE ITS system to test its effectiveness for secure and instructor-regulated learning *without* degrading instructional value. We analyze: (i) *Response Quality*: Does the system actually guide the student through the learning process, or does it simply give away the answer? We check if the LLM adheres to the curriculum and offers hints instead of solutions, (ii)

Safety and Security: Can the system be tricked? We test if students can bypass the rules to access hidden exam keys or content they are not supposed to see yet, and (iii) *System Performance*: Is the system optimized enough for real-time user interaction? In all experiments, SAGE is compared against two state-of-the-art commercial LLM baselines (GPT and Gemini). These two LLMs are selected as they are two of the top models in the LLM leaderboard [19].

A. Experimental Setup

We evaluate along three modalities. First, a controlled *simulation harness* generates synthetic learners with known ground-truth mastery: each learner is assigned a latent per-tier mastery vector, and responses are sampled with tier-appropriate guess and slip rates. We distinguish *balanced* learners, whose mastery is uniform across tiers, from *lopsided* learners, who are strong on declarative recall but weak on applied competence the profile the Evidence Diversity Problem concerns. Second, we analyze *telemetry* from a live deployment in an introductory C-programming course [PLACEHOLDER: $N \approx \dots$ students, IRB #...], including an independent held-out assessment (Sec. IV-D) that the learner model never observes. Third, we retain the C-EduBench benchmark and an expanded adversarial suite for the inherited-gate evaluation (Sec. IV-H). Unless stated otherwise, simulation results report the mean over [PLACEHOLDER: S] seeds with 95% confidence intervals, and paired comparisons use McNemar’s test.

B. Metrics

The metrics that we use to evaluate responses from SAGE and baseline LLM-based AI-Tutor systems are as follows: *a) Code Density*: The fraction of response content that appears as executable code (e.g., code blocks or code-like lines). Lower values indicate the tutor avoids dumping full solutions and instead emphasizes guidance. *b) Security Compliance*: The percentage of Security-category benchmark prompts for which the system refuses or safely redirects disallowed requests. *c) Curriculum Compliance*: The percentage of queries for which the response respects course constraints such as locked topics, prerequisites, and instructor-defined pacing. *d) Pedagogy Score (1–5)*: A rubric-based rating where a score of 5 reflects stepwise scaffolding and a score of 1 reflects direct solution dumping. *e) Overall Judge Preference*: The percentage of benchmark items for which an LLM-based judge [20] selects a system response as the best according to a fixed rubric emphasizing helpfulness for novice learners, pedagogical guidance, and safe refusal when applicable.

C. Over-Crediting Resistance

Headline study. Run lopsided non-masters (strong recall, weak applied) through a single-scalar BKT and the factored model; show the baseline certifies many and the factored

model certifies none, while both still certify true masters. Empirically realizes Theorem 1. (Runnable now.)

D. Certification Validity Against Held-Out Assessment

Check certification against an external test the model never saw; show certified learners outperform non-certified learners on it. Establishes the mastery signal is real, not just internally consistent. (Needs classroom.)

E. Decision Quality: Deferral versus Misclassification

Separate the two error types; show errors concentrate on deferral (safe: ask for more evidence) rather than false certification (unsafe: admit an unready learner). (Simulation part runnable now.)

F. Model Calibration

Per-tier Brier scores and reliability diagrams from the prediction log; show predicted probabilities match observed success rates. (Runnable now, existing data.)

G. Robustness to Parameter Misspecification

Sweep guess/slip rates across a plausible range; show certification stays well-behaved throughout, defending against the “you tuned it” objection. (Runnable now.)

H. Inherited Gate: Compliance and Adversarial Robustness

Carried-over conference evaluation: the C-EduBench comparison table (strengthened with CIs) and the expanded red-team audit reconciling the security number, with residual failures shown to be generative-layer, not access-control bypass. Framed as a secondary no-regression check. (Runnable now.)

I. System Performance

Latency of the learner-model update alongside gatekeeper overhead and time-to-first-token; confirm the added component keeps interaction real-time. (Runnable now.)

J. Usability in Authentic Use

SUS plus instructor dashboard-actionability and learner helpfulness ratings from the deployment, with brief qualitative feedback. (Needs classroom.)

V. CONCLUSION

REFERENCES

- [1] Enkelejda Kasneci, Katharina Sessler, Stefan Küchemann, Maria Bannert, Daria Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günemann, Sebastian Krusche, Gitta Kutyniok, Josef Nerb, and Gjergji Kasneci. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023.
- [2] Rong Wu and Zhonggen Yu. Do ai chatbots improve students' learning outcomes? evidence from a meta-analysis. *British Journal of Educational Technology*, 55(1):10–33, 2024.
- [3] Barry J. Zimmerman. Becoming a self-regulated learner: An overview. *Theory Into Practice*, 41(2):64–70, 2002.
- [4] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISec '23, pages 79–90, New York, NY, USA, 2023. Association for Computing Machinery.
- [6] Ernesto Panadero. A review of self-regulated learning: Six models and four directions for research. *Frontiers in psychology*, 8:422, 2017.
- [7] Clayton Cohn, Surya Rayala, Namrata Srivastava, Joyce Horn Fontelles, Shruti Jain, Xinying Luo, Divya Mereddy, Naveeduddin Mohammed, and Gautam Biswas. A theory of adaptive scaffolding for LLM-based pedagogical agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 1757–1765, 2026.
- [8] Vincent Aleven, Ido Roll, Bruce M. McLaren, and Kenneth R. Koedinger. Help helps, but only so much: Research on help seeking with intelligent tutoring systems. *International Journal of Artificial Intelligence in Education*, 26:205–223, 2016.
- [9] Roger Azevedo and Allyson F. Hadwin. Scaffolding self-regulated learning and metacognition—implications for the design of computer-based scaffolds. *Instructional Science*, 33(5):367–379, 2005.
- [10] Yuhong Shi, Kun Yu, Yifei Dong, and Fang Chen. Large language models in education: A systematic review of empirical applications, benefits, and challenges. *Computers and Education: Artificial Intelligence*, 10:100529, 2026.
- [11] Upasana Roy, Vani Seth, Mahady Hasan Rayhan, Ashish Pandey, Mauro Lemus Alarcon, Matthew Maschmann, and Prasad Calyam. Closed-loop cnt growth using retrieval augmented generation and human feedback. In *2025 IEEE 5th International Conference on Human-Machine Systems (ICHMS)*, pages 128–133, 2025.
- [12] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [13] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *ACM Trans. Inf. Syst.*, 44(2), December 2025.
- [14] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.
- [15] Chroma Developers. Chroma: The open-source embedding database, 2023. Accessed: 2026-01-14.
- [16] Neo4j, Inc. Neo4j graph database, 2007. Accessed: 2026-01-14.
- [17] OpenAI. Chatgpt. <https://chatgpt.com/>, 2026. Accessed: 2026-01-28.
- [18] Google. Gemini. <https://gemini.google.com/>, 2026. Accessed: 2026-01-28.
- [19] Artificial Analysis. Llm leaderboard - comparison of over 100 ai models from openai, google, deepseek & others, 2026. Accessed: 2026-01-15.
- [20] Ying Sheng Siyuan Zhuang Zhanghao Wu Yonghao Zhuang Zi Lin Zhuohan Li Dacheng Li Eric P. Xing Hao Zhang Joseph E. Gonzalez Ion Stoica Lianmin Zheng, Wei-Lin Chiang. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.